

☺ 计算机网络实验报告二

网页服务器实战和应用层协议分析

学号:2013365 姓名:颜铭

1. 实验准备

任务要求:

搭建个人网页的前后端服务器。支持基本网页介绍和音频嵌入

使用wireshark进行抓包测试, 分析http协议结构

1.1 实验条件

本次网页使用本地回环(loopback)端口为3000,使用 `express.js` 搭建服务器应用框架。

在 `chrome` 浏览器上运行

使用 `wireshark` 软件进行抓包处理

2. 实验原理

1. Http协议概览:

以HTTP1.1协议为例子, 它是用于互联网上的超文本传输的协议。它是HTTP/1.0的升级版, 引入了许多新特性和改进, 最大改进是引入了持久性连接。HTTP 1.1定义了客户端和服务端之间的通信规则, 包括请求和响应的结构。

这里的http1.0是最初的互联网协议, 主要包含请求和响应的报文, 通过表头和解析行和头的日期服务器码和内容接收码等内容工作。

HTTP 1.1的主要特性和改进:

- 持久连接 (Persistent Connections)**: HTTP 1.1引入了持久连接, 使得客户端和服务端之间可以在单个TCP连接上发送多个请求和响应, 而不需要每次请求都建立新的连接。这提高了性能和效率。
- 管道化 (Pipeline)**: HTTP 1.1允许多个请求并行发送, 而不必等待先前的请求的响应。这减少了延迟并提高了吞吐量。

3. **增量响应 (Incremental Response)**：服务器可以发送部分响应而不必等待完整的响应构建完成，从而更快地开始传输数据。
4. **主机头 (Host Header)**：HTTP 1.1引入了"Host"请求头，允许一个HTTP服务器上托管多个域名，因此同一台服务器可以处理多个不同域名的请求。
5. **分块传输编码 (Chunked Transfer Encoding)**：HTTP 1.1支持分块传输编码，允许服务器将响应分成多个块，并将每个块独立传输，以便更早地开始处理。

- **请求行**：包括HTTP方法（GET、POST、等）、请求的URL和HTTP版本。

例如：

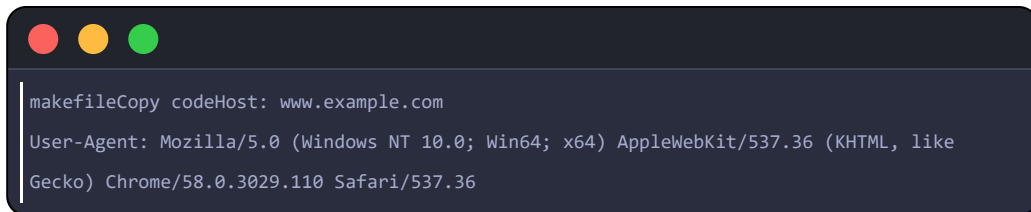


```
bashCopy code
GET /index.html HTTP/1.1
```

Fence 2-1

- **请求头部**：包含请求的元数据，如主机信息、用户代理、Cookie等。

例如：



```
makefileCopy code
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/58.0.3029.110 Safari/537.36
```

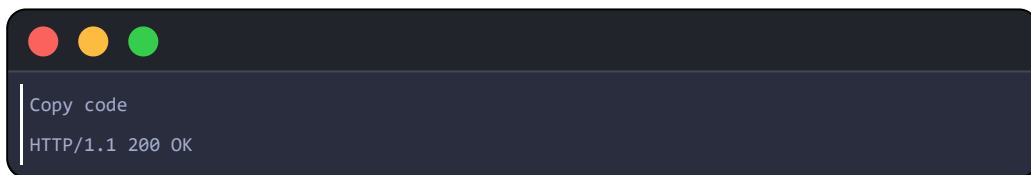
Fence 2-2

- **空行**：空行分隔请求头部和请求主体。
- **请求主体**（可选）：包含请求的数据，如POST请求中的表单数据。

HTTP 1.1 响应报文的结构：

HTTP 1.1响应报文由以下部分组成：

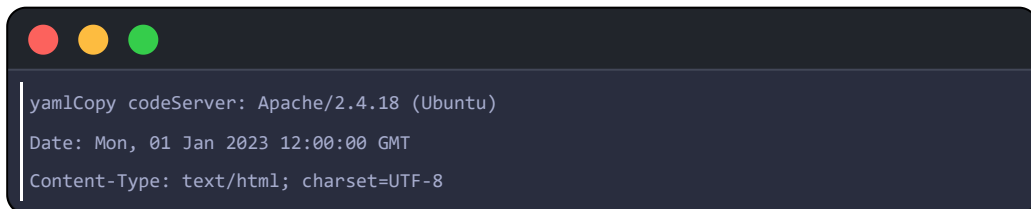
- **状态行**：包括HTTP版本、状态码和状态消息。例如：



```
Copy code
HTTP/1.1 200 OK
```

Fence 2-3

- **响应头部**：包含响应的元数据，如服务器信息、日期、内容类型等。例如：



```
yamlCopy code
Server: Apache/2.4.18 (Ubuntu)
Date: Mon, 01 Jan 2023 12:00:00 GMT
Content-Type: text/html; charset=UTF-8
```

Fence 2-4

- **空行**：空行分隔响应头部和响应主体。
- **响应主体**：包含响应的实际数据，如HTML页面、图像、JSON等。

HTTP 1.1的请求和响应报文结构是基本的构建块，用于在客户端和服务端之间传输数据。这些报文允许Web应用程序在互联网上传递超文本内容和其他资源。

HTTP 1.1 请求报文的结构：

HTTP 1.1响应报文由以下部分组成：

- **状态行**：包括HTTP版本、状态码和状态消息。例如：

```
Copy code
HTTP/1.1 200 OK
```

Fence 2-5

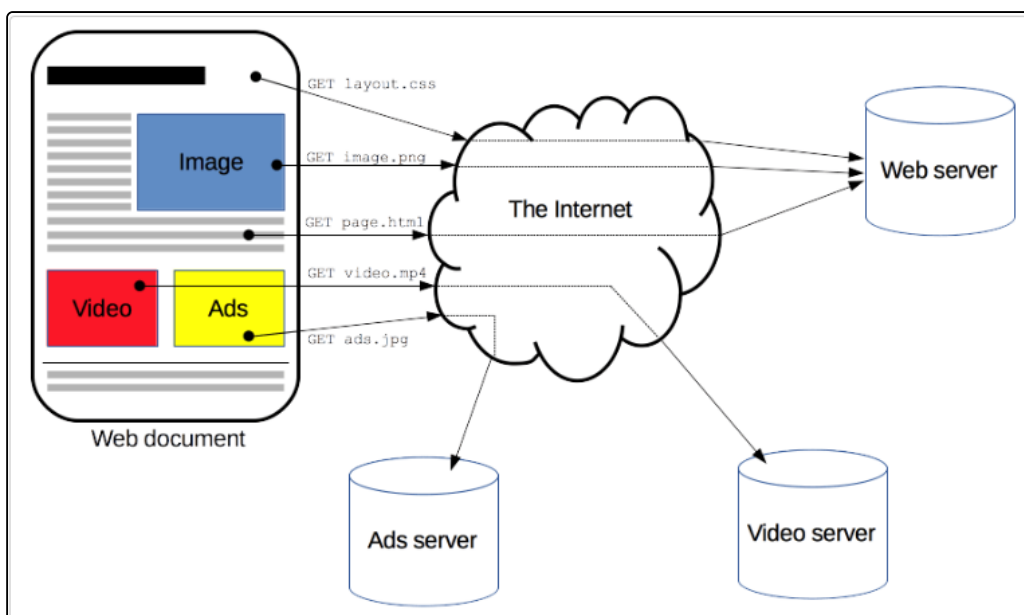
- **响应头部**：包含响应的元数据，如服务器信息、日期、内容类型等。例如：

```
Copy code
Server: Apache/2.4.18 (Ubuntu)
Date: Mon, 01 Jan 2023 12:00:00 GMT
Content-Type: text/html; charset=UTF-8
```

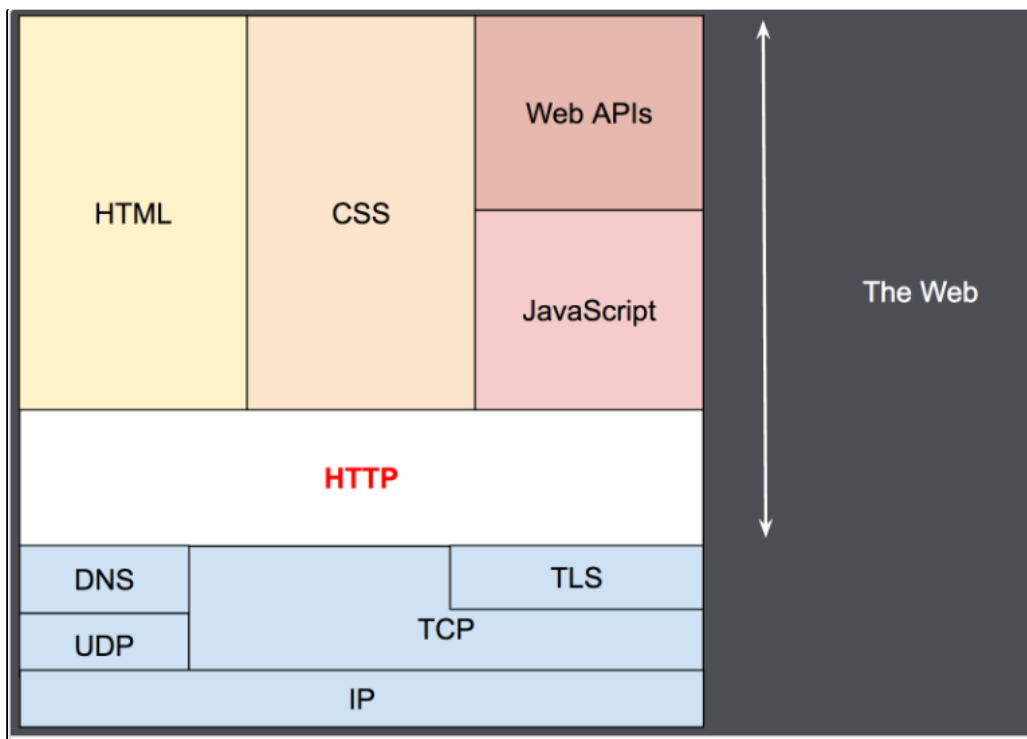
Fence 2-6

- **空行**：空行分隔响应头部和响应主体。
- **响应主体**：包含响应的实际数据，如HTML页面、图像、JSON等。

在web段有用户前端向服务器发送的工作流程如下：



搭配高级web编程语言进行开发的协议结构为：



2. tcp传输层协议的解包关键字

TCP 是一种面向连接的可靠的传输层协议，它使用三次握手和四次挥手的过程来建立和断开连接。以下是这两个过程的简要说明：

○ 三次握手

：用于建立连接，其过程如下：

1. 客户端向服务器发送一个 SYN 报文，表示请求建立连接，并携带一个初始序列号 x 。
2. 服务器收到 SYN 报文后，回复一个 SYN+ACK 报文，表示同意建立连接，并携带一个确认序号 $x+1$ 和一个初始序列号 y 。
3. 客户端收到 SYN+ACK 报文后，回复一个 ACK 报文，表示确认建立连接，并携带一个确认序号 $y+1$ 。至此，连接建立成功。

○ 四次挥手

：用于断开连接，其过程如下：

1. 客户端向服务器发送一个 FIN 报文，表示请求断开连接，并携带一个序列号 m 。
2. 服务器收到 FIN 报文后，回复一个 ACK 报文，表示确认收到请求，并携带一个确认序号 $m+1$ 。此时，客户端到服务器的方向已经断开。
3. 服务器向客户端发送一个 FIN 报文，表示也请求断开连接，并携带一个序列号 n 。
4. 客户端收到 FIN 报文后，回复一个 ACK 报文，表示确认收到请求，并携带一个确认序号 $n+1$ 。此时，服务器到客户端的方向也已经断开。至此，连接断开成功。

3. 现代的网页开发主流使用的 `html+css+js` 作为网页控制语言

- **html** 是浏览器解析和渲染的主体超文本标记语言对象，这里面又包含了 `fetch API` `jquery` `dom` 容器等技术。
- **css** 为层叠样式表，可以控制html排版和风格类型，提供了简介的对html模块化管理功能
- **javascript** 是一种解释脚本语言，受浏览器引擎原生驱动，具有很完备的node.js模块为核心的生态和后端方法

3. 使用过程

3.1 搭建个人的网页

实验中使用express.js`生态搭建个人网页的后端，包括了监听端口开启本地回环服务，以及

首先我们启动在本地回环 `localhost` 端口的3000服务为：

```
app.listen(port,()=>{
  console.log(`running on http://localhost:${port}`);
});
```

Fence 3-1

初始化静态资源和使用目录为

```
const app = express();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const port=3000;
app.use(express.static(path.join(__dirname, 'www')));

app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());
```

Fence 3-2

1. `GET` 请求的数据发送与状态控制响应,包括刷新页面前后的数据保存，可以被报文响应头部解析为`==GET==`
2. `POST` 请求的数据是提交表单描述的一种用户交互式行为后，用于向服务器提交数据。被报文响应头部解析为`==POST==`

我们给出express的代码，注意每一种请求都需要保证对应路由 `url` 的唯一性，因此我们有GET请求

```
app.get('/', function (req, res) {
  res.sendFile(path.join(__dirname, 'www', 'webYumIndex.html'))
});
app.get('/refresh', (req, res) => {
  res.redirect('/');
});
app.get("/msg", (req, res) => {
  res.json({ message: "你好欢迎来到Yum的网页" });
});
```

Fence 3-3

在网页还设置了留言功能，可以将POST请求发送到服务器的文本数据保存到本地并查看(缺少用户端)

```

app.post('/comment', (req, res) => {
  const msg = req.body.message;
  fs.appendFile('./comments.txt', msg + '\n', (err) => {
    if (err) {
      throw err;
    }
    console.log('saved comment on comments.txt');
    res.redirect('/');
  })
});

```

Fence 3-4

网页内容是由html和css控制的，这里选择了按钮，表单，输入框，背景图片，嵌入音频等基本个人网页内容的需求，大部分动态资源如音频因为不需要记录是由html嵌入javascript控制的

其中音频嵌入的html代码为

我们给出html和css部分代码

```

<audio src="./audio/output.mp3" style="margin-left: 100px;margin-top: 100px"
type="audio/mpeg" id="audio-source" controls>
</audio>

```

Fence 3-5

完整的html网页特例化代码为

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>yum 的个人网页(连入express后端编写的web服务器)</title>

  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
  <link rel="stylesheet" href="./css/webYumStyle.css">
  <script src="https://code.jquery.com/jquery-3.6.3.min.js"></script>
</head>
<body>

<h1>学号:2013365 姓名:颜铭 专业:智能科学与技术</h1>

<div>
  欢迎来到yum的个人网页
</div>

<br>
<br>

<p style="background: cyan;font-style: italic;font-size: 15pt;
font-weight: bold;margin-left: 20%; margin-right: 20%;align-content: center;border-width: 5pt;
border-style: solid" id="message" >
  消息框:
</p>

<span id="unlockMsg" style="width:75px;height: 5px;font-size: 10pt;
font-weight: bolder;background-color:rgba(240,255,255,0.4);margin-left: 45%">
  滑动以解锁
<script>
  var slider = document.getElementById("myRange");
  var unlockMsg = document.getElementById("unlockMsg");

```

```

        slider.oninput = function() {
            if(slider.value==100){
                unlockMsg.innerHTML = "解锁成功!";
                // var p = document.getElementById('message').textContent = "消息框:欢迎, 新的访客";

                fetch('/msg')
                    .then(response=>{
                        if(!response.ok)
                            throw new Error('msg missed!!!');
                        window.location.reload();
                    }).then(data =>{
                        let hello_msg = document.getElementById('message');
                        hello_msg.textContent = JSON.stringify(data['message']);
                    }).catch(error => console.error('Error:', error));
            }
        }
    }

</script>
</span>
<input type="range" min="1" max="100" value="20" class="slider" id="myRange" style="margin-left: 45%">

<br>
<p>
    <audio src="./audio/output.mp3" style="margin-left: 100px;margin-top: 100px"
    type="audio/mpeg" id="audio-source" controls>

</audio>
<progress id="progressBar" value="0" max="1" style="margin-left: 100px;margin-top: 150px;visibility: hidden"></progress>
<button id="player-audio" class="myButton">
    <i class="fa fa-play" aria-hidden="true"></i>
    播放音频
</button>
<button id="stop-audio" class="myButton">
    <i class="fa fa-pause" aria-hidden="true"></i>
    暂停音频
</button>
<button id="reload-audio" class="myButton">
    <i class="fa fa-circle" aria-hidden="true"></i>
    刷新音频
</button>
<script>
    const audioSource = document.getElementById('audio-source');
    const playerButton = document.getElementById('player-audio');
    const stopButton = document.getElementById('stop-audio');
    const reloadButton= document.getElementById('reload-audio');
    const progressBar = document.getElementById('progressBar');

    audioSource.addEventListener('timeupdate',function (){
        if(audioSource.duration){
            progressBar.value = audioSource.currentTime / audioSource.duration;
        }
    });
    audioSource.addEventListener('ended', function() {
        progressBar.value = 0;
    });
    playerButton.addEventListener('click',()=>{
        audioSource.play();
    });
    stopButton.addEventListener('click',()=>{
        audioSource.pause();
    });
    reloadButton.addEventListener('click',()=>{
        audioSource.update();
    });
</script>
</p>

```

```

<br>
<br>
<select style="text-align: center;font-size: 15pt;margin-right: 100px;color: skyblue;font-family: SimHei">
  <option value="default">默认选项</option>
  <option value="other">其他选项</option>
</select>

<p style="background: rgba(230,235,198,0.5);width:400px;height: 100px;column-count: 2;margin-left:10% ">
  <button id="update" style="margin-left: 200px;font-size: 12pt;width: 120px">刷新这个界面
</button>
  <script>

    const elementButton = document.getElementById('update');

    elementButton.addEventListener('click',function () {
      alert("成功刷新界面");
      fetch('/refresh')
        .then(response=>{
          if(!response.ok)
            throw new Error('network response error cause not ok!!!');
        }).catch(error => console.error('Error: ',error));

    });

  </script>
</p>
<script>window.onload = function (){
  if(window.location=== '/comment'){
    alert('消息接受成功')
  }
}</script>

<form action="/comment" method="POST" style="margin-right: 50px">
  <p style="width: 300px;height: 20px;text-emphasis-style: double-circle;margin-left: 70%">留言区: 8#x2728 </p>
  <input type="text" name="message" style="margin-left: 70%">
  <button style="margin-left: 70%" type="submit">提交留言</button>
</form>

</body>
</html>

```

Fence 3-6

以及部分css代码（动画效果）

```

@-webkit-keyframes myAnimationFirst{
  0% {background-color: azure;left: 50px; top: 10px;width: 200px;height: 10px}
  33% {background-color: cyan;left: 250px; top: 110px;width: 200px;height: 10px}
  66% {background-color: salmon;left: 450px; top: 10px;width: 200px;height: 10px}
  100% {background-color: springgreen;left: 50px; top: 10px;width: 200px;height: 10px}
}

```

Fence 3-7

3.1.1 网页运行结果截图



需要说明的是个人网页功能比较简陋。

3.2 wireshark抓包实验

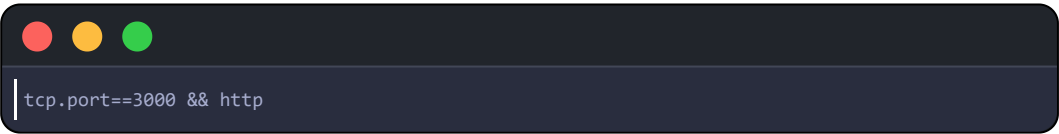
基本操作。

- 1. 安装wireshark并勾选pcap以及对应inet拓展模块的安装选项
- 2. 打开wireshark并点击启用抓包服务范围 `loopback`



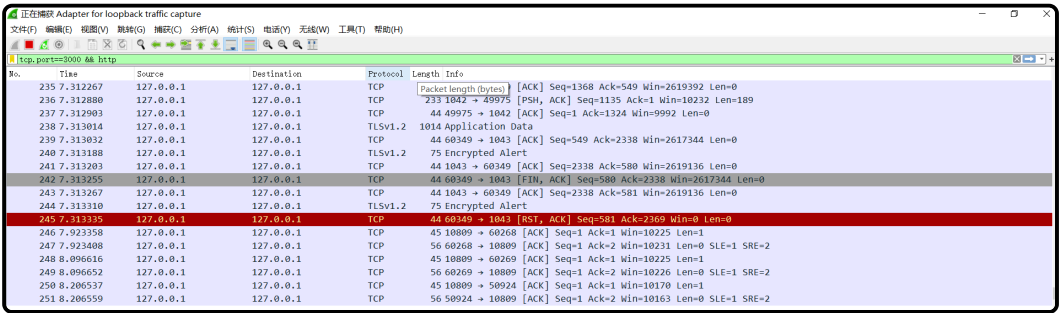
- 3. 选择并应用过滤器并开始捕获记录:

过滤规则为:



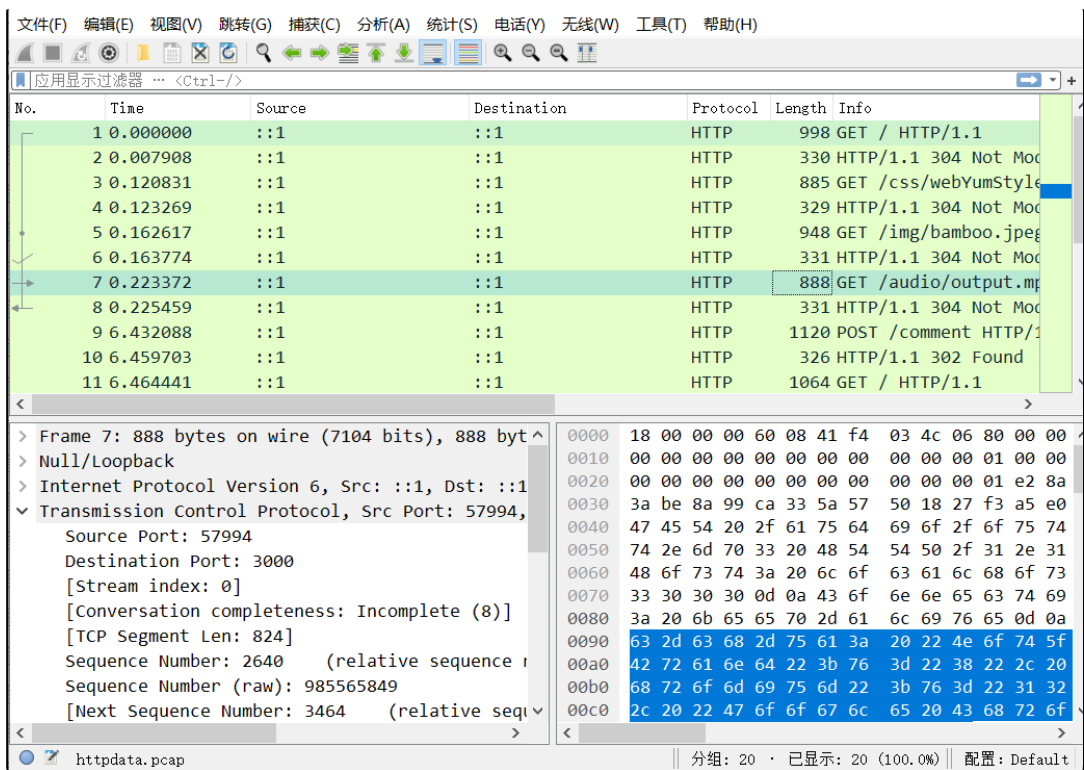
Fence 3-8

表示本地回环tcp服务的3000端口上的http解析数据包



- 4. 我们依次播放音频，刷新页面，滑动条，输入留言并提交

可以得到如下的wireshark抓包



3.3 抓包分析

以音频为例可以解包得到如下数据为



对应到上面的请求状态行，解析到了方法为GET,并且传输文件为mp3音频文件，http请求为1.1版本

此外也解析了主机 Host 的IP地址

最后也对 fetch 方法解析了html网页同源的初始点到目的路由 /audio

由于 express 中没有设置 cookie，因此我们可以得到的是webstorm的用户信息。最后还关注了audio刷新进度条的帧数，定位在1s左右(实际播放时间)

4. 实验分析和总结

通过组件化接触前沿的后端部署知识，使用html描述语言模拟了请求和响应，并学会了通过wireshark进行简单的抓包分析

展望和不足

由于没有定制客户端，功能实现比较基础，整体还是静态资源占比大。没有更加利用js搭建分布式web服务器，后续计划更加学习将个人网页部署到博客网站上。

5. 助教提出问题课后补充回答

1. Q: 为什么TCP断开连接是要四次挥手? 为什么有些四次挥手会出现三次挥手的现象,具体表现是什么呢?

A: 由于服务端和客户端运行在各自机器上, 对内存操作和端口访问等通信基本要求是双向的, 那么就是双方都可以断开主动连接。这一个发送关闭信号必须返回接受的报文作为确认, 也就是说必须双方都必须严格确认达成协议。这样至多两次的应答只能让服务器请求方进入等待关闭状态, 也即是模拟最少过程为:

- 客户端打算关闭连接, 此时会发送一个 TCP 首部 `FIN` 标志位被置为 `1` 的报文, 也即 `FIN` 报文, 之后客户端进入 `FIN_WAIT_1` 状态。
- 服务端收到该报文后, 就向客户端发送 `ACK` 应答报文, 接着服务端进入 `CLOSED_WAIT` 状态。
- 客户端收到服务端的 `ACK` 应答报文后, 之后进入 `FIN_WAIT_2` 状态。等待服务端处理完数据后, 也向客户端发送 `FIN` 报文, 之后服务端进入 `LAST_ACK` 状态。这里明显是要处理关闭任务导致了异步操作。
- 客户端收到服务端的 `FIN` 报文后, 回一个 `ACK` 应答报文, 之后进入 `TIME_WAIT` 状态
- 服务器收到了 `ACK` 应答报文后, 就进入了 `CLOSE` 状态, 至此服务端已经完成连接的关闭。

`FIN => ACK (WAIT) <= FIN => ACK (SUCCESS)`

这样每个方向都要一个 `FIN` 和 `ACK`, 因此被称为四次挥手

至于为什么有些四次挥手会变为三次挥手, 查阅资料是由于服务器框架在第一次ACK后不会立刻返回 `FIN`, 而是等待处理完线程任务后没有多余数据需要处理, 那么再同第三次请求一同发送给客户端确认WAIT即可。

具体表现就是wireshark抓包中的延迟确认机制。

1048	12.549266	:::1	:::1	TCP	64 55485 → 3000 [ACK] Seq=3464 Ack=1066 Win=2617856 Len=0
2364	17.565252	:::1	:::1	TCP	64 3000 → 55485 [FIN, ACK] Seq=1066 Ack=3464 Win=2616320 Len=0
2365	17.565319	:::1	:::1	TCP	64 55485 → 3000 [ACK] Seq=3464 Ack=1067 Win=2617856 Len=0
2562	20.232539	:::1	:::1	TCP	64 55485 → 3000 [FIN, ACK] Seq=3464 Ack=1067 Win=2617856 Len=0

中间的 `[FIN,ACK]`

解包后有如下关键部分

Flags: 0x011 (FIN, ACK)	
000.	 = Reserved: Not set
...0	 = Accurate ECN: Not set
.... 0...	 = Congestion Window Reduced: Not set
.... .0..	 = ECN-Echo: Not set
.... ..0.	 = Urgent: Not set
.... ...1	 = Acknowledgment: Set
.... 0...	 = Push: Not set
....0..	 = Reset: Not set
....0.	 = Syn: Not set
>1	 = Fin: Set

2. Q: 为什么四次挥手后最后服务器需要等待2MSL呢

A: 这里的MLS指的是一次报文在链路上停留的最长时间, 这样处理是为了防止有额外的至多一条的等待确认连接任务在服务器停止完全前无法处理。也是为了使得出现丢包情况, 导致最后一次发送给服务器的确认在网络丢失导致了客户端关闭但是服务器端关闭这样的资源浪费的场景。